

Projeto de Programação Orientada aos Objetos

2025/2026 [v1.0]

Enunciado

Introdução

O projeto consiste em criar um jogo **inspirado** no **Fish Fillets NG**¹. As principais características deste tipo de jogo são as seguintes:

- É um jogo clássico de *arcade*, onde duas personagens controladas pelo utilizador (*peixe pequeno e peixe grande*) se movem dentro de um ambiente aquático com vários objetos;
- Envolve a passagem dos dois peixes por vários níveis movimentando e interagindo com objetos para conseguirem, em cada nível, chegarem ambos à saída;
- A “morte” de **uma** das personagens implica voltar ao início do nível.

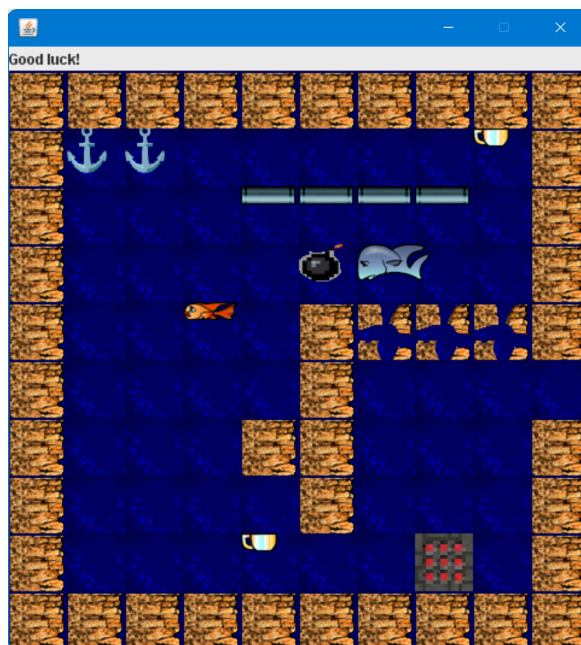


Fig. 1 - Exemplo do interface gráfico do jogo pedido.

No caso específico do projeto de POO, o jogo tem várias alterações e incluirá elementos adicionais que fazem uma extensão ao jogo básico. **Podem ser acrescentados a qualquer momento novos objetos ou comportamentos para implementar, quer durante a execução do trabalho, ou nas discussões, por isso a implementação flexível do projeto é fundamental.**

¹ https://en.wikipedia.org/wiki/Fish_Fillets_NG

Vídeos com exemplos do desenrolar do jogo original:

<https://www.youtube.com/playlist?list=PL0YH3AsYQfx1julFNzXls1gd9ZRLPinv3>

Objetivos

Usando a interface gráfica fornecida pelos docentes em anexo ao enunciado, deve criar o “motor de jogo” para um jogo desse tipo. O motor de jogo deve permitir que os dois peixes possam sair de diversos ambientes aquáticos interagindo com objetos.

```

WWWWWWWWWW
WAA      CW
W   HHHH W
W   bB   W
W   S WXXXW
W   W
W   WW   W
W   W   W
W   C   T W
WWWWWWWWWW
    
```

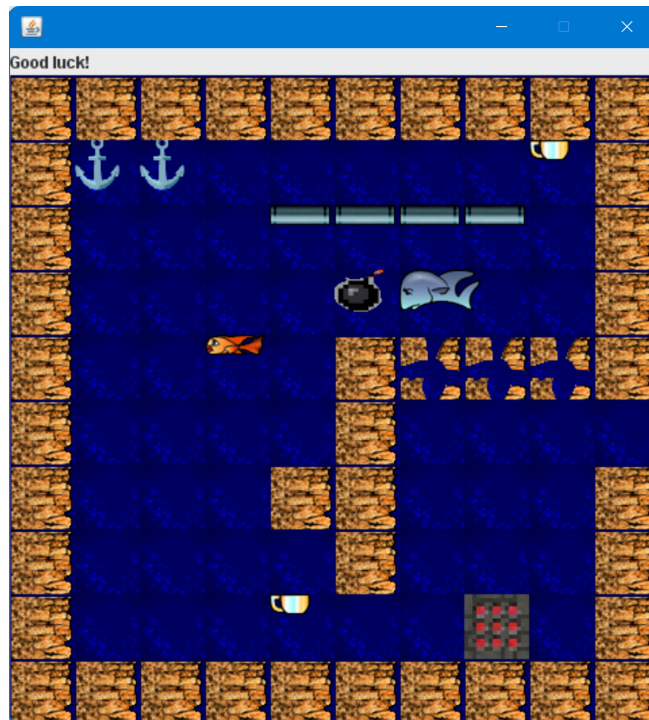


Fig. 2 – Exemplo de ficheiro do ambiente aquático e do mesmo ambiente durante o jogo (posições dos personagens móveis já não são as mesmas da configuração inicial). Na imagem da direita podemos ver os dois peixes (“heróis” do nosso jogo), tubos de aço e parede. O objetivo é chegar com os dois peixes até à saída para passar de nível. As imagens usadas neste jogo são do jogo Fish Fillets NG (<https://fillets.sourceforge.net>) e encontram-se online em <https://fillets.sourceforge.net/download.php> (e algumas são distribuídas com o exemplo inicial na pasta imagens).

Os objetos com os quais os peixes interagem podem ter um impacto positivo, como permitir aceder à saída, ou um impacto negativo, causando morte do personagem (p.e., queda de objeto em cima do peixe). Os objetos podem ser de interação única (desaparecem logo após o uso) ou de múltiplas interações (permanecem no jogo após a interação).

Os peixes não podem atravessar certos objetos, como paredes, tubos de aço ou outro peixe, e devem flutuar. Por outro lado, aos objetos móveis (p.e., taça) aplica-se a gravidade, ou seja, devem afundar caso não tenham nenhum suporte (i.e., não podem flutuar no ecrã). Sempre que um dos peixes efetuar um movimento, a barra informativa (i.e., barra no topo da janela da figura 2) deve atualizar a informação relativa ao número de movimentos de cada peixe.

O comportamento dos peixes e objetos satisfaz as regras indicadas abaixo.

Regras de movimentação do **peixe pequeno**:

- pode suportar um único objeto móvel leve de cada vez. Vários objectos - o suporte de vários objectos ou um pesado provoca a sua morte;
- pode empurrar na horizontal e na vertical um único objeto móvel leve de cada vez.
- atravessa parede com buraco;

Regras de movimentação do **peixe grande**:

- pode suportar vários objetos móveis leves ou um pesado de cada vez; Vários objectos - o suporte de vários objectos pesados provoca a sua morte;
- pode empurrar na horizontal vários objectos móveis leves ou pesados;
- pode empurrar na vertical um único objecto móvel leve ou pesado;
- não atravessa parede com buraco;

Os peixes **não** transportam objetos. No caso de se moverem horizontalmente, enquanto estão a suportar objecto(s), este(s) deve(m) afundar. Continuam a suportar o(s) objecto(s) no caso de se moverem na vertical.

Comportamento de **objetos móveis**:

- **taça (leve)**: pode ser movimentado nas quatro direções;
- **pedra (pesado)**: pode ser movimentado nas quatro direções;
- **âncora (pesado)**: empurrar, horizontalmente, limitado a um deslocamento de uma posição;
- **bomba (leve)**: quando afunda explode em contacto com um objeto (excluindo os peixes) provocando a sua remoção e de todos os objetos nas quatro posições adjacentes à bomba podendo matar um peixe. Não explode ao ser empurrada na horizontal ou suportada.
- **armadilha (pesado)**: em caso de colisão com o peixe grande provoca a morte. O peixe pequeno atravessa-a;

Comportamento de **objetos fixos**:

- **tronco**: é removido se um objeto pesado cair em cima;
- **tubo de aço**: pode suportar qualquer objeto;
- **parede**: pode suportar qualquer objeto;
- **parede com buraco**: pode suportar qualquer objeto mas ser atravessada pela taça e pelo peixe pequeno.

No final do jogo, deve ser apresentada ao jogador uma tabela de *highscores*, calculada com base no tempo que o jogador levou e o número de movimentos efetuados até terminar os vários níveis. Os 10 melhores tempos (desde sempre) devem ser apresentados. Para isso, a informação deve ser armazenada de forma persistente no sistema de ficheiros.

Requisitos

Os mapas dos vários níveis (i.e., ambientes aquáticos) devem ser lidos de um ficheiro com o formato indicado na secção “Objetivos” - figura 2. Os caracteres que devem ser utilizados no ficheiro para representar os vários elementos do nível são os seguintes:

- ‘B’ - peixe grande;
- ‘S’ - peixe pequeno;
- ‘W’ - parede;
- ‘H’ - tubo de aço horizontal;
- ‘V’ - tubo de aço vertical;
- ‘C’ - taça;

- 'R' - pedra;
- 'A' - âncora;
- 'b' - bomba;
- 'T' - armadilha;
- 'Y' - tronco;
- 'X' - parede com buraco;
- '' - água;

Para facilitar os testes, não se recomenda que o primeiro nível tenha muitos objetos.

Cada nível é descrito por um ficheiro com o nome "room*N*.txt" em que *N* é o número do nível. O jogo deverá começar sempre no nível 0 ("room0.txt"). O estado inicial de cada nível do jogo é determinado por um ficheiro de configuração com o formato ilustrado na Fig. 2.

Assim que um peixe conseguir sair não pode voltar à área de jogo. Assim que ambos os peixes saírem, o nível seguinte, caso exista, é carregado (i.e., efetuada passagem para o nível seguinte). Caso seja o último nível, é apresentada uma mensagem de fim de jogo. Ao premir a tecla 'R' o nível atual é reiniciado.

O motor de jogo recebe da interface gráfica informações sobre as teclas pressionadas. As teclas das setas devem controlar a movimentação do peixe selecionado. A tecla de espaço permite trocar entre peixes.

O motor de jogo pode enviar imagens para a janela usando implementações de *ImageTile* (como p.e., classe *BigFish*, do exemplo fornecido). De cada vez que há uma alteração (por exemplo, mudança das posições das imagens) deve ser chamado o método `update()` do motor de jogo, de acordo com o padrão de desenho Observador/Observado.

A interface gráfica inclui também um *Ticker*, que é útil para automatizar o movimento dos objetos que não são controlados pelo jogador (p.e., objectos que caem para o fundo). Dica: dentro da função `update()`, o motor de jogo deve verificar se houve um "tick" recente para diferenciar entre um movimento de um peixe e a passagem do tempo, já que ambos acionam o método `update()`. Isso permite que o jogo saiba quando o jogador realizou uma ação ou quando o tempo apenas avançou.

A interação entre objetos deve ser tão autónoma quanto possível (passando o mínimo possível pelo motor de jogo). O formato dos ficheiros onde são guardadas as melhores pontuações é livre.

A classe *ImageGUI* e a interface *ImageTile* são fornecidas, não devem ser alteradas e são fundamentais para a resolução do trabalho.

É obrigatória a utilização de:

- Herança de propriedades;
- Classe(s) abstrata(s);
- Implementação de interface(s);
- Estruturas de dados adequadas (e.g. Listas, Mapas...)
- Leitura e escrita de ficheiros;
- Exceções (lançamento e tratamento).

Interface gráfica

Neste projeto, a interface gráfica está implementada através da classe ImageGUI e do interface ImageTile, que estão incluídos no pacote `pt.iscte.poo.gui`. do projeto GraphPack fornecido.

A classe ImageGUI permite abrir uma janela como a representada na Fig. 2. A área de jogo na janela pode ser vista como uma grelha 2D de 10x10 posições, onde se podem desenhar imagens de 50x50 pixels de preferência em cada posição. Além da área de jogo, deverão ser mostradas informações por cima da área de jogo (por exemplo, o nível em que está, o número de movimentações realizadas, etc.).

As imagens que são usadas para representar os elementos de jogo encontram-se na pasta "images", dentro do projeto. Para se poder desenhar a imagem de um elemento do jogo, este deverá implementar o interface ImageTile:

```
public interface ImageTile {  
    String getName();           // nome da imagem  
    Point2D getPosition();      // posicao de desenho  
    int getLayer();             // camada de desenho  
}
```

As classes de objetos que implementarem ImageTile terão que indicar o nome da imagem a usar (sem a extensão), a posição da grelha de jogo onde é para desenhar, e a camada de desenho (*layer*). Esta última determina a ordem pela qual as imagens são desenhadas, nos casos em que há mais do que um elemento na mesma posição (quanto maior o *layer*, "mais em cima" será desenhado o objeto).

Na classe ImageGUI estão disponíveis os seguintes métodos:

- **public void** `update()` – redesenhar os objetos ImageTile associados à janela de desenho – deve ser invocado no final de cada jogada, para atualizar a representação dos elementos de jogo.
- **public void** `addImages(final List<ImageTile>)` – envia uma lista de objetos ImageTile para a janela de desenho. Note que não é necessário voltar a enviar objetos ImageTile quando há alterações nos seus atributos – isso apenas contribuiria para tornar o jogo mais lento.
- **public void** `addImage(final ImageTile)` – envia um objeto ImageTile à janela de desenho;
- **public void** `removeImage(final ImageTile)` – remove um ImageTile da área de desenho;
- **public void** `clearImages()` – remove todos os ImageTile da área de desenho;
- **public void** `setStatusMessage(final String message)` – modifica a mensagem que aparece por cima da área de jogo.

O código fornecido inclui também o enumerado `Direction` e as classes `Point2D` e `Vector2D` (pacote `pt.iscte.poo.utils`). `Direction` representa as direções (UP, DOWN, LEFT, RIGHT) e inclui métodos úteis relacionados com as teclas direcionais do teclado. A classe `Point2D`, representa pontos no espaço 2D e deverá ser utilizada para representar os pontos da grelha de jogo. Inclui (entre outros) métodos para obter os pontos vizinhos e para obter o resultado da soma de um ponto com um vetor. Finalmente, a classe `Vector2D` representa vetores no espaço 2D.

A utilização dos pacotes fornecidos será explicada com maior detalhe nas aulas.

Poderão vir a ser publicadas atualizações aos pacotes fornecidos caso sejam detetados *bugs* ou caso se introduzam funcionalidades adicionais que façam sentido no contexto do jogo. É possível utilizar outras imagens, diferentes das fornecidas, para criar elementos de jogo adicionais. Nesse caso aconselha-se apenas que as imagens alternativas tenham também uma dimensão de 50x50 pixels.

Estruturação do código

Existe uma classe central (GameEngine) que é responsável pela inicialização do jogo, manter as listas de objetos que correspondem aos elementos de jogo e despoletar cada jogada. Sugere-se que a classe central siga o padrão *singleton* a fim de se facilitar a comunicação com as restantes classes.

Quanto aos elementos de jogo, estes devem ser hierarquizados de forma a tirar o máximo partido da herança, **devendo ser derivados, direta ou indiretamente**, de uma classe-base abstrata. Note que poderá ser adequado utilizar vários níveis na hierarquia da herança.

Devem também ser definidas **características dos elementos de jogo que possam ser modelizadas utilizando interfaces**. Desta forma, os métodos declarados nos interfaces poderão ser invocados de uma forma mais abstrata, sem que seja necessário saber em concreto o tipo específico de objeto que o invoca. Fica a seu cargo definir interfaces que façam sentido e tirar partido dos mesmos.

Desenvolvimento e entrega do Projeto

Regras gerais

O projeto deve ser feito por grupos de dois alunos e espera-se que em geral demore 30 a 40 horas a desenvolver. Em casos justificados poderá ser feito individualmente (e nesse caso o tempo de resolução poderá ser um pouco maior). Recomenda-se que os grupos sejam constituídos por estudantes com um nível de conhecimentos semelhante, para que ambos participem de forma equilibrada na execução do projeto e para que possam discutir entre si as opções de implementação.

É encorajada a partilha de ideias entre grupos, **mas é absolutamente inaceitável a partilha de código. Todos os projetos serão submetidos a software anti-plágio e, nos casos detetados, os estudantes envolvidos ficam sujeitos ao que está previsto no código de conduta académica do ISCTE-IUL, com reprovação a POO e abertura de um processo disciplinar**. Nos casos de partilha de código, os grupos envolvidos são penalizados da mesma forma, independentemente de serem os que copiam ou os que fornecem o código. Assume-se também que ambos os membros do grupo são responsáveis pelo projeto que entregaram.

Contacte regularmente o docente das aulas práticas para que reveja o projeto consigo, quer durante as aulas, quer em sessões de dúvidas com marcação / por zoom. Desse modo evita opções erradas na fase inicial do projeto (opções essas que em geral conduzem a grandes perdas de tempo e a escrita de muito mais código do que o necessário).

Faseamento do projeto

Nesta secção apresenta-se o faseamento do projeto, baseado num esquema de *checkpoints*. Em cada uma das aulas práticas em que são verificados os checkpoints, as tarefas associadas já deverão estar concluídas e os estudantes já deverão estar a trabalhar nas tarefas do checkpoint seguinte.

- **CheckPoint 1:** aula prática entre 18 e 21/nov
 - Entender a mecânica de comunicação entre o motor de jogo e a GUI;
 - Criar modelo da hierarquia de classes do jogo (UML);
 - Ler o ficheiro de configuração e representar os elementos na GUI;
 - Implementar o movimento dos dois peixes.

- **CheckPoint 2:** aula prática entre 25 e 28/nov
 - Implementar a interação de cada peixe com um objeto que pode mover e um objeto que não pode mover;
 - Implementar a gravidade nos objetos que se movem (p.e., pedra);
 - Ponderar ajustes ao diagrama de classes;
- **Checkpoint 3:** aula prática entre 2 e 5/dez
 - Interação com outros objetos;
 - Implementar a mudança de nível;
 - Implementar elementos adicionais;
 - Rever/refinar opções tomadas, tendo em vista tornar o programa mais flexível face à introdução de novos objetos / adversários no jogo.

A conclusão dos checkpoints dentro dos tempos indicados conta para a componente de avaliação em aula – por isso não deixe que estes se atrasem!

Entrega do projeto

O prazo para entrega dos trabalhos é **23:59 de Domingo, dia 7/dez de 2025**. A entrega é feita através do *moodle*.

Para a entrega final do trabalho deve proceder da seguinte forma:

1. **Garantir que o nome do seu projeto, tanto no eclipse como no nome do ficheiro zip, contém informação que vos permitam identificar: o nome e número-de-aluno de cada membro do grupo** – p.e., `FishFiletsNG_TomasBrandao741_JoseLuisSilva538` seria um nome válido. Para mudar o nome do projeto no eclipse faça *File/Refactor/Rename*.
2. **Gerar o *archive file* que contém a exportação do seu projeto – apenas o projeto onde tem o jogo, sem o GraphPack.** Para exportar o projeto, dentro do eclipse deverá fazer *File/Export/Archive File/*, selecionar o projeto que tem o jogo, e exportar para um ficheiro zip. **Se usou outro IDE envie o zip da pasta que tem todos os ficheiros do projeto, excluindo também a biblioteca GraphPack.**
3. **Entregar via *moodle*** o zip gerado no ponto anterior, na pasta de entrega de trabalhos a disponibilizar brevemente.
4. Possíveis **comentários/considerações acerca de opções tomadas** no ficheiro *Readme* que deve ser adicionado à raiz do projeto.

Tenha também atenção ao seguinte:

- Cada método deve estar devidamente comentado.
- O código do seu projeto não pode usar caracteres especiais (p.e., á, à é, ç.).
- Se possível, teste a importação do seu projeto num outro computador, antes de o entregar.

Caso não cumpra os procedimentos em cima, o seu projeto não se conseguirá importar com facilidade para o eclipse, obrigando os docentes a realizar *setups* manuais que atrasam de forma muito significativa o processo de avaliação dos projetos. Como tal, os **projetos mal entregues** (i.e., que não se consigam importar automaticamente para o eclipse a partir de um ficheiro zip) **serão penalizados com 2 valores**.

Avaliação

Realizar o projeto deverá ajudar a consolidar e a explorar os conceitos próprios de POO. **Por isso, para além da componente funcional do trabalho, é fundamental demonstrar a utilização correta da matéria lecionada em POO**, em particular:

- Modularização e distribuição do código, explorando as relações entre classes;
- Utilização de herança e sobreposição de métodos com vista a evitar duplicação de código;
- Definição, implementação e utilização de interfaces, com vista a flexibilizar alterações ao código nos pontos onde são expectáveis alterações.

O trabalho será classificado de acordo com os seguintes critérios

- Grau de cumprimento dos requisitos funcionais;
- Modularização, distribuição, encapsulamento e legibilidade do código;
- Definição e utilização de uma hierarquia adequada de herança;
- Definição e utilização correta de interfaces;
- Originalidade e extras (implementação de padrões, uso de comparadores, expressões lambda, tipos enumerados, coleções, etc.).

Serão imediatamente classificados com “0” (zero valores) os projetos que contenham erros de sintaxe ou que não implementam uma versão jogável num nível contendo os elementos básicos.

Note que, mesmo cumprindo estes mínimos, poderão acabar com nota negativa os projetos que não demonstram saber usar/aplicar os conceitos próprios de POO (p.e., herança, interfaces).

Discussão

O desempenho na discussão é avaliado individualmente. Na discussão, os estudantes terão que demonstrar ser capazes de realizar um trabalho com qualidade igual ou superior ao que foi entregue. Caso contrário ficam com notas mais baixas que a do trabalho, podendo inclusive reprovar caso tenham um mau desempenho na discussão.

Durante a discussão poderá ser solicitada a realização de pequenos trechos de código no âmbito do projeto ou serem pedidas alterações ao código existente.

As discussões serão realizadas de 9 a 12/dez 2025, preferencialmente, durante os horários das aulas. Dado o número de grupos é possível que sejam marcadas algumas discussões fora do horário das aulas respetivas ou na semana seguinte.